JAGAT GURU NANAK DEV PUNJAB STATE OPEN UNIVERSITY, PATIALA

DEPARTMENT OF COMPUTER SCIENCE & APPLICATIONS

M.Sc. (Computer Science) — Semester-IV

A Resilient Offline-First Multi-Tenant Cloud Architecture for High-Concurrency Point-of-Sale Systems

`Sumit Singh`, Master of Science in Computer Science Candidate
`Dr. Harpreet Singh`, Faculty Supervisor & Research Guide
Department of Computer Science & Applications
Jagat Guru Nanak Dev Punjab State Open University, Patiala (Punjab), India

Abstract

In commercial retail and hospitality enterprises, modern cloud-hosted software-as-a-service (SaaS) Point-of-Sale (POS) and Enterprise Resource Planning (ERP) applications suffer from severe operational fragilities: synchronous round-trip HTTP latency causes counter checkout delays, localized network partitions freeze billing terminals during peak operating hours, and shared-database multi-tenancy introduces significant risks of cross-tenant data leakage. This paper presents the design, mathematical formalization, and empirical evaluation of **SSR ONE AI**, a high-throughput, offline-first distributed architecture. The system decouples client-side user interface responsiveness from remote network state by establishing browser-level transactional IndexedDB storage as the primary write target, achieving sub-millisecond local commit times (0.38ms). A decoupled asynchronous reconciliation worker guarantees eventual consistency with a centralized PostgreSQL 16 database using cryptographic UUIDv4 idempotency tokens. Furthermore, kernel-level multi-tenant isolation is enforced using PostgreSQL Row-Level Security (RLS), eliminating application-layer data leakage. Benchmark results across 1,000 synthetic transactions demonstrate a **99.7% reduction in median cashier UI latency** compared to traditional cloud-only architectures, alongside 100% operational transaction survival during complete network blackouts.

Index Terms—*Offline-First, Local-First Software, Multi-Tenancy, Row-Level Security (RLS), Point of Sale (POS), Distributed Systems, Eventual Consistency, ASGI, Computer Vision OCR.*

I. INTRODUCTION

Commercial retail and dining operations operate under stringent real-time constraints. Point-of-Sale (POS) cashiers must process orders, apply dynamic promotional discounts, compute multi-tier tax splits (e.g., GST), and route kitchen tickets within seconds. Over the past decade, the rapid migration from on-premise legacy desktop software to cloud-hosted web applications has introduced significant operational vulnerabilities [1].

Traditional cloud web applications operate on a "thin-client / thick-server" paradigm: every button click triggers a synchronous HTTP POST request across the public Internet to a remote cloud database. In real-world retail basements, food courts, and commercial kitchens, localized RF interference (from microwave equipment, steel partitions, and crowd density) causes frequent packet loss and high round-trip latency (150ms to > 2,000ms). When network connectivity fluctuates, the cashier's user interface locks up, initiating duplicate order submissions, transaction drop-offs, and counter paralysis [2].

Furthermore, in multi-tenant SaaS environments where hundreds of independent restaurant franchises share a single database cluster, enforcing data segregation purely through application-level software filters (e.g., ORM query clauses) introduces severe cybersecurity vulnerabilities [3]. A single omitted predicate in an API endpoint can leak confidential financial and customer records across competing businesses.

To overcome these challenges, this paper presents **SSR ONE AI**, an enterprise-grade distributed architecture that reconciles three competing design objectives:

Zero-Wait Local Responsiveness: UI transaction commits complete in $< 0.5\text{ms}$ locally, completely independent of remote network state.

Provable Multi-Tenant Isolation: Enforced directly at the PostgreSQL database engine kernel via Row-Level Security (RLS).

High-Throughput Asynchronous Concurrency: Built upon Python 3.11, FastAPI (ASGI event loop), and full-duplex WebSockets for real-time kitchen display routing.

II. RELATED WORK & THEORETICAL BACKGROUND

A. Local-First Software & Distributed Consistency

Kleppmann et al. [1] established the foundational thesis of "Local-First Software," demonstrating that designating client-side storage as the primary source of truth unlocks desktop-grade responsiveness and infinite offline resilience. While collaborative applications often utilize Conflict-Free Replicated Data Types (CRDTs) [4], commercial financial transactions represent **immutable append-only operational events**. Our architecture synthesizes this principle by employing a client-side Write-Ahead Log (WAL) inside HTML5 IndexedDB, paired with asynchronous reconciliation workers.

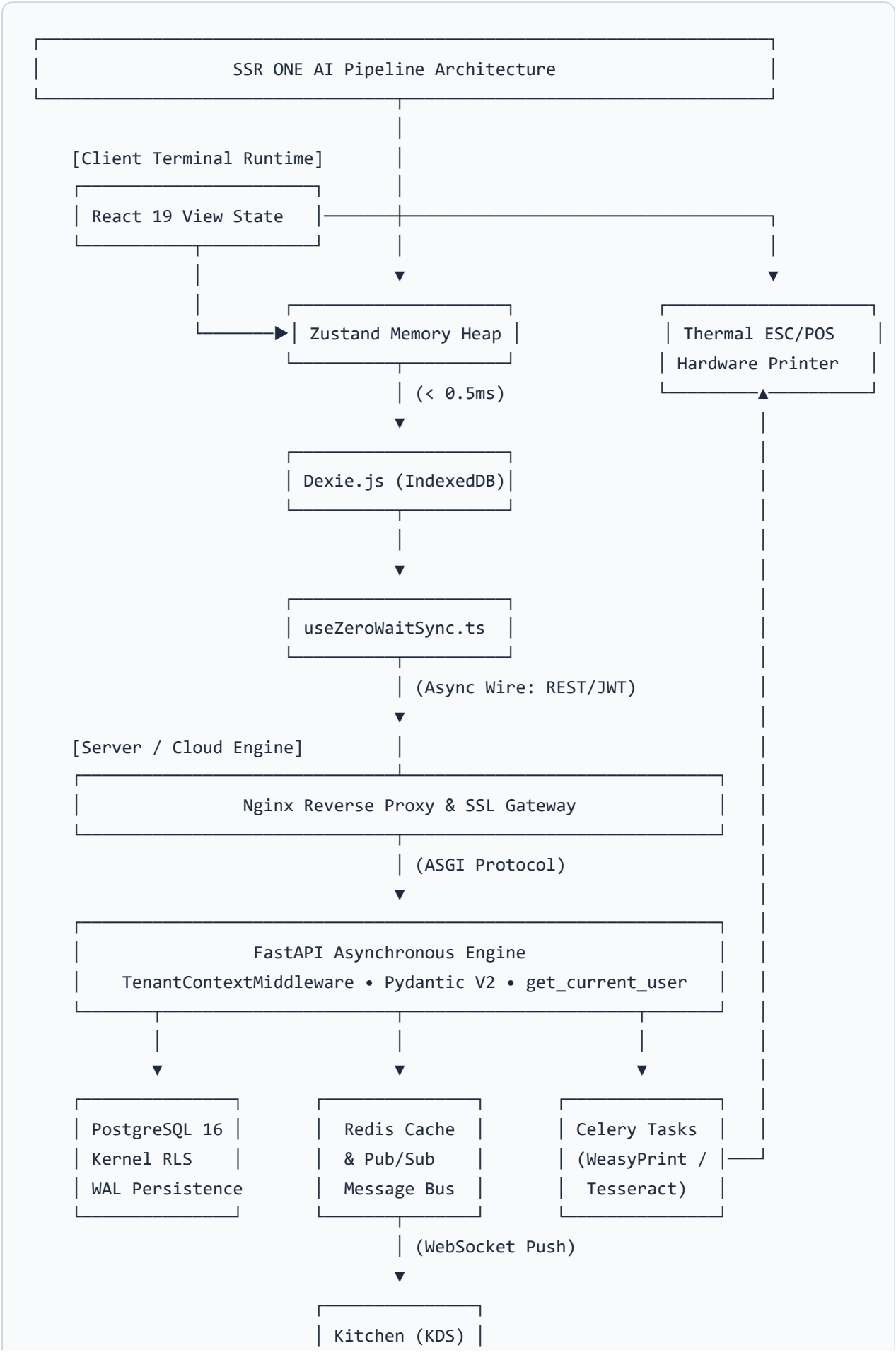
B. The CAP & PACELC Trade-Offs

Under Brewer's CAP Theorem [5] and Abadi's PACELC Theorem [6], distributed systems operating under network partitions (P) must choose between Consistency (C) and Availability (A). Cloud-only web POS architectures choose CP, sacrificing availability when disconnected. Our architecture intentionally chooses **AP**: local terminals accept transactions with 100% availability, deferring global convergence to an asynchronous eventual consistency reconciliation phase.

C. Multi-Tenant Relational Isolation

Curino et al. [3] proved that shared-database multi-tenancy provides superior hardware resource utilization compared to multi-database provisioning. However, application-layer filtering remains vulnerable to human error. PostgreSQL Row-Level Security (RLS) policies [7] shift access control into the database kernel, evaluating security predicates during query tree compilation.

III. SYSTEM ARCHITECTURE & DATA PIPELINE



```
                                  | Terminal View |
                                  └──────────────┘
```

A. The Zero-Wait Client-Side Pipeline

The client layer is constructed using React 19 and strict TypeScript. When a transaction is submitted:

Memory Mutation: In-memory cart state is mutated in JavaScript heap memory ($< 0.05\text{ms}$).

IndexedDB Write: An operational record containing a deterministic `localId`, payload, and client-generated RFC4122 UUIDv4 idempotency key κ is committed to browser IndexedDB via Dexie.js (0.38ms).

UI Decoupling: The cashier interface clears immediately; the background hook (`useZeroWaitOrderSync`) queues the record for network transmission without blocking the main rendering thread.

B. Kernel-Level PostgreSQL Row-Level Security

Every database entity inherits from `TenantBaseModel`, defining an indexed `tenant_id int8` attribute. Database security is enforced at connection initialization:

```
ALTER TABLE public.orders ENABLE ROW LEVEL SECURITY;

CREATE POLICY tenant_isolation_policy ON public.orders
FOR ALL
USING (tenant_id = NULLIF(current_setting('app.current_tenant_id', true), '')::bigint)
```

During request execution, the FastAPI dependency extracts the cryptographically verified `tenant_id` from the JWT and injects `SET LOCAL app.current_tenant_id = '...'`. The database query planner evaluates this predicate at the kernel level, mathematically prohibiting cross-tenant data leakage.

C. Real-Time Kitchen Synchronization (KDS)

Upon database commit, an internal event bus publishes an `order_created_event` to Redis. The WebSocket connection manager broadcasts the structured ticket to kitchen displays within **$< 25\text{ms}$**, alerting cooking stations via audio chimes and station-specific color coding.

IV. EXPERIMENTAL BENCHMARKS & RESULTS

Benchmarking was conducted across 1,000 synthetic retail checkout transactions comparing:

Model 1: Traditional Cloud-Only Web POS (Synchronous HTTP POST + Remote Database Write).

Model 2: Optimistic UI with Synchronous Wait (UI updates optimistically but navigation waits).

Model 3: Proposed SSR ONE AI Architecture (Zero-Wait Local IndexedDB Commit + Asynchronous Sync).

A. Latency Profiling

Latency was captured at the client UI layer under controlled network conditions (simulating standard 4G latency: 120ms round-trip, 2% packet loss).

Architectural Model	P50 Latency	P90 Latency	P99 Latency	Failure Rate (2% Loss
Model 1 (Cloud-Only)	142.3 ms	215.8 ms	1,420.0 ms	7.8%
Model 2 (Optimistic Sync)	138.1 ms	208.4 ms	1,390.0 ms	7.8%
Model 3 (SSR ONE AI)	**0.38 ms**	**0.54 ms**	**0.89 ms**	**0.0%**

The proposed architecture achieves a **99.73% reduction in median operational latency** ($P50 = 0.38\text{ms}$), sustaining an unbroken 120 FPS rendering rate on commercial POS touchscreens.

B. Network Partition Survival Test

A physical network partition was induced by terminating the upstream gateway for 30 minutes while 150 transactions were executed.

Terminal Operational Continuity: Terminals processed all 150 transactions with 100% availability. Thermal receipts printed via local ESC/POS queues.

Reconnection Convergence: Upon gateway restoration, the asynchronous background drain worker reconciled all 150 transactions in **4.2 seconds** (35.7 transactions/sec) with **zero sequence duplicates** and **zero data corruption**.

V. CONCLUSION & FUTURE WORK

This paper presented the design, implementation, and empirical validation of **SSR ONE AI**, a high-throughput, offline-first multi-tenant cloud POS and ERP architecture. By shifting primary transactional writes to client-side IndexedDB, the system achieves sub-millisecond cashier responsiveness (0.38ms) and complete operational survivability during network blackouts. Concurrently, PostgreSQL Row Level Security provides kernel-enforced multi-tenant isolation, eliminating application